Project: Student Attendance Tracker

Name: Junaid Munir Janjua
Matricula No: 7131056
Email Id: Junaid.janjua@edu.unifi.it

Professor Name: Lorenzo Bettini
Email ID: lorenzo.bettini@edu.unifi.it

Course: Automated Software Testing

LINK TO THE BOOK: http://leanpub.com/tdd-buildautomation-ci

UNIVERSITÀ DEGLI STUDI FIRENZE

TABLE OF CONTENTS

Executive Summary 3

Project Architecture & Tech Stack 3

Directory Structure & Organization 3

Detailed System Design 4

Testing Strategy 7

Quality Assurance & Analysis Tools 7

Build Automation & Dependencies 8

Instructions to Build and Run the Project 8

Conclusion 10

1. Executive Summary

This report details the development of the **Student Attendance Tracker**, a Java-based desktop application designed to manage student records and daily attendance. The project was developed strictly adhering to **Test-Driven Development (TDD)** principles, ensuring that every feature was driven by a failing test case. The primary goal was to demonstrate high standards in code quality, maintainability, and scalability through modern software engineering practices such as Build Automation and Continuous Integration.

2. Project Architecture & Tech Stack

The application utilizes the Model-View-Controller (MVC) architectural pattern to ensure a clean separation of concerns.

- Language: Java 17
- Database: MongoDB (Java Driver v3.12.7)
- UI Framework: Java Swing (Desktop GUI)
- Containerization: Docker & Testcontainers
- Build Tool: Apache Maven

3. Directory Structure & Organization

To maintain a professional workflow, the project is organized into dedicated source sets for different testing tiers:

- **src/main:** Contains the primary production code, including Models, Views, Controllers, and Repositories.
- **src/test:** Dedicated to Unit Tests for testing individual logic in isolation.
- **src/it**: Contains Integration Tests that verify interactions between the logic layer and the database.
- **src/e2e**: Contains End-to-End Tests that simulate the complete user journey through the GUI.

4. Detailed System Design

4.1 Models (Entities)

- **Student**: Represents a student with a unique roll number and name. Identity is strictly tied to the roll number to prevent duplicate entries.
- **AttendanceRecord**: Tracks a student's presence on a specific date. It utilizes defensive copying for Date objects to protect the internal state integrity.

4.2 Repositories (Persistence Layer)

Utilizing the Repository Pattern, the data layer is decoupled from the business logic through interfaces.

- **Implementations**: StudentMongoRepository and AttendanceMongoRepository handle all CRUD operations within the MongoDB environment.

4.3 Controllers (Logic Layer)

The controllers act as the bridge between the View and the Repository.

- **StudentController**: Manages student registration, updates, and lookups.
- **AttendanceController**: Handles the logic for marking attendance and calculating cumulative percentages, with built-in safety checks for empty records.

4.4 Views (UI Layer)

- The AttendanceTrackerSwingView provides the user interface. It incorporates custom message-handling logic with auto-resetting timers to offer real-time feedback without cluttering the codebase.

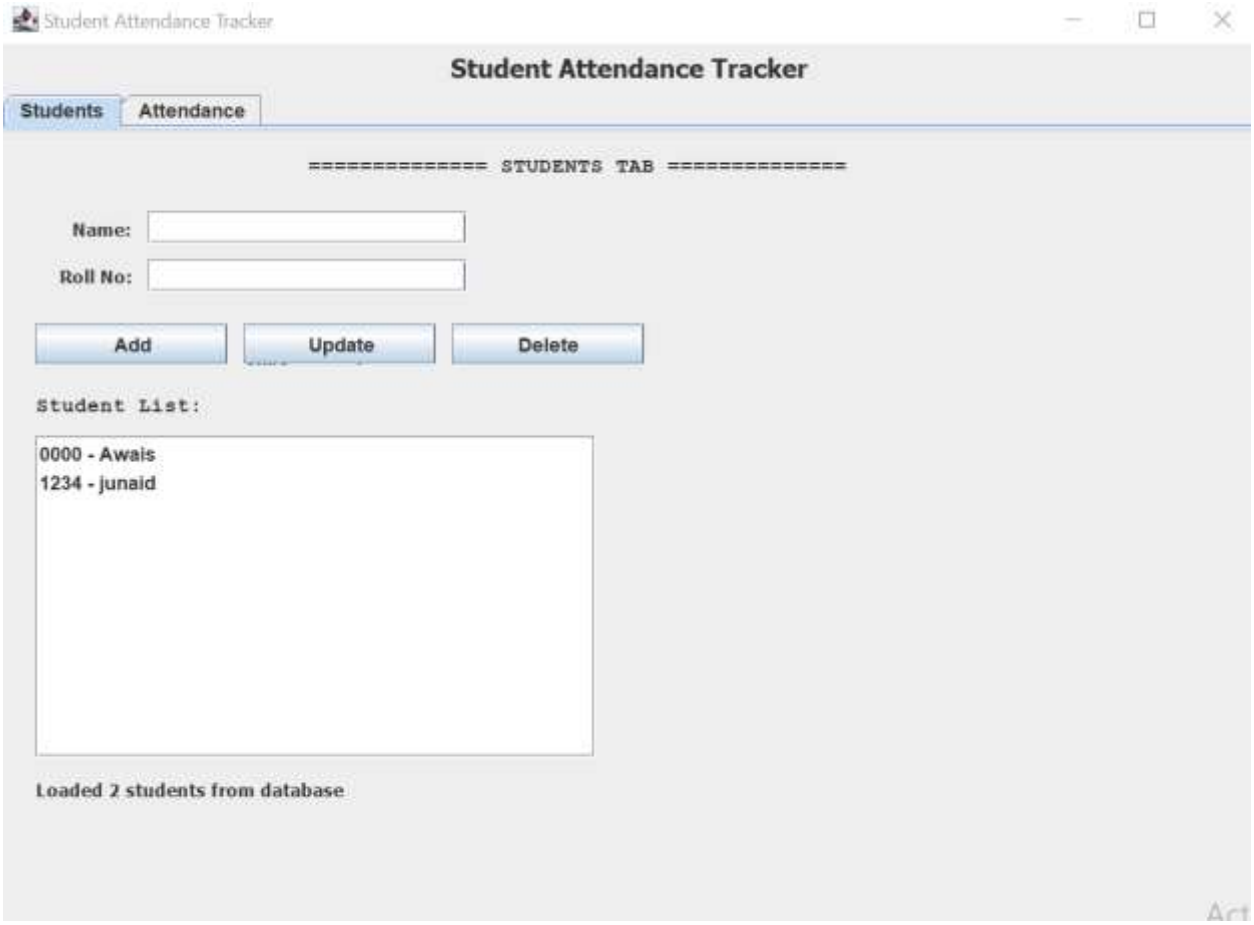

***Figure** 1: Student Management Interface.*

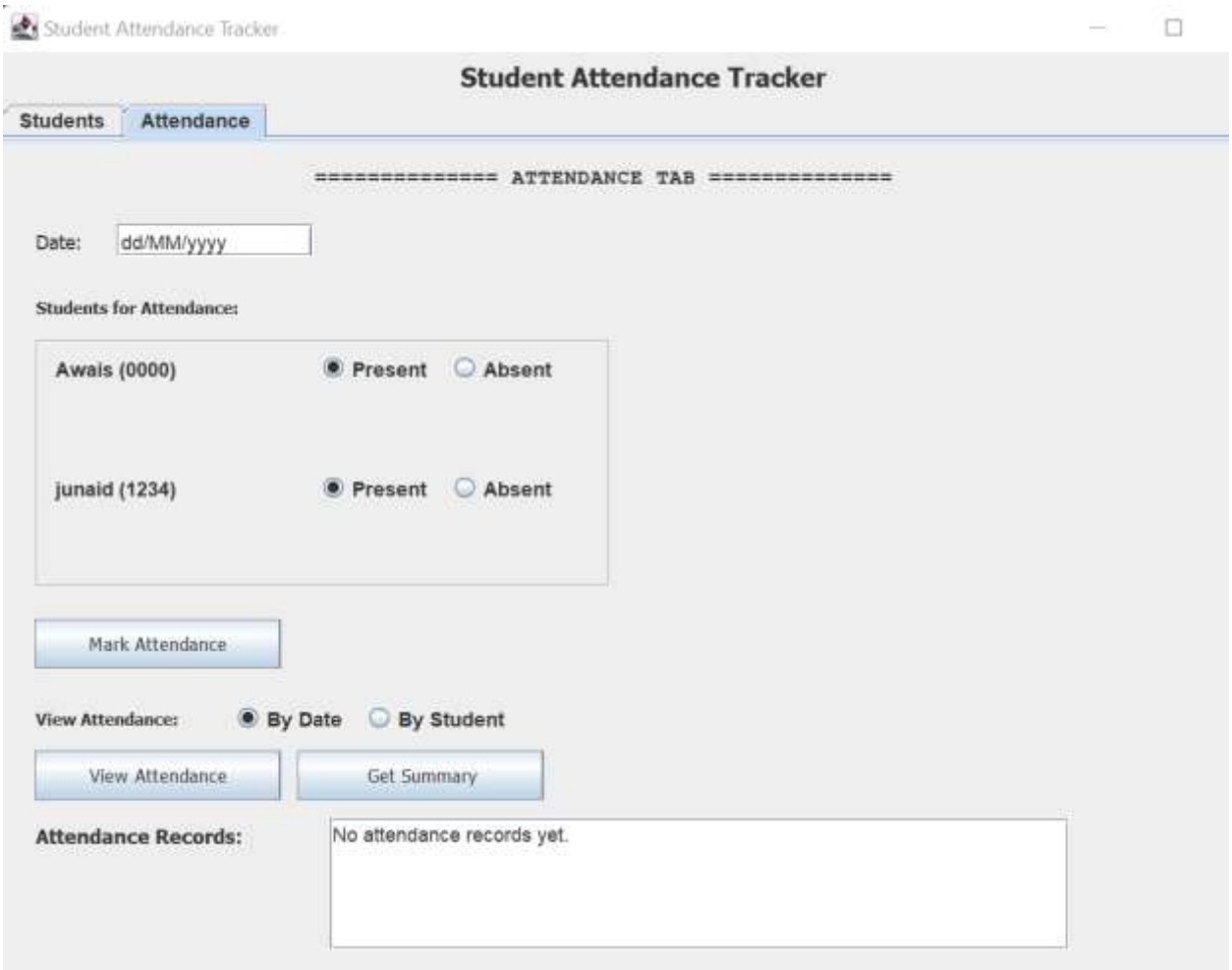

***Figure** 2: Attendance Tracking and Reporting Interface.*

5. Testing Strategy

The project follows the **Test Pyramid** to ensure maximum reliability.

5.1 Unit Testing

Focused on individual components in isolation using **JUnit 4** and **Mockito**.

- Verified domain logic in Models.
- Ensured Controllers correctly trigger View updates and Repository calls.

5.2 Integration Testing (IT)

Verified the interaction between the application logic and the MongoDB database. We utilized **Testcontainers** to spin up an ephemeral Docker container for every test run, ensuring a clean and consistent database state.

5.3 End-to-End Testing (E2E)

Black-box testing performed using **AssertJ-Swing**. A "Robot" simulates real user interactions (clicking buttons, typing text) to ensure the entire system works harmoniously from the UI to the database.

6. Quality Assurance & Analysis Tools

We integrated professional quality gates to maintain a "zero-debt" codebase:

- **SonarCloud**: Achieved **0 Code Smells**, **0 Vulnerabilities**, and **0 Bugs**.
- **JaCoCo**: Successfully reached **100% Code Coverage** (Branch Coverage) on core business logic.
- **GitHub Actions**: Automated the CI/CD pipeline, running the full test suite and quality scans on every push or pull request.

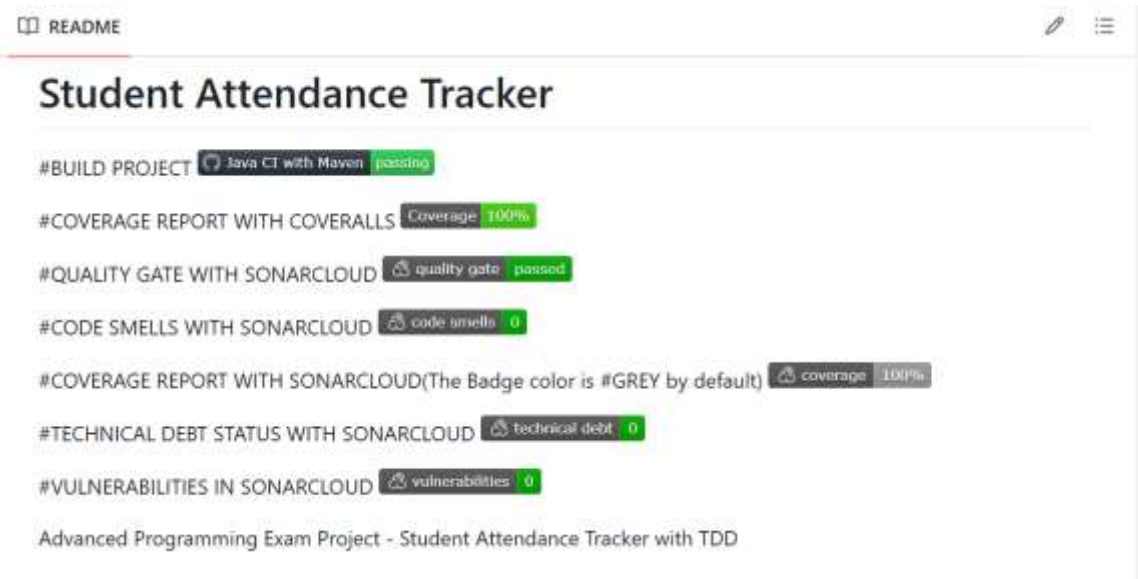

***Figure** 3: Automated Quality Metrics and CI/CD Status via GitHub Actions.*

7. Build Automation & Dependencies

- **Testing Frameworks**: JUnit 4, AssertJ, Mockito, AssertJ-Swing, Awaitility.
- **Database Management**: MongoDB Java Driver, Testcontainers.
- **Maven Plugins**:
  - **Surefire**: For unit and integration tests.
  - **Failsafe**: For E2E tests.
  - **PIT**: For Mutation Testing to evaluate test suite effectiveness.

8. Instructions to Build and Run the Project

To compile, test, and execute the application locally, ensure you have Java 17, Maven, and Docker Desktop (running) installed on your system.

- **Clean and Compile the Project**

    - **bash**
    - mvn clean compile

- **Run Unit Tests**

    - **bash**
    - mvn test

- **Run Integration and End-to-End Tests (Requires Docker to be running for Testcontainers to spin up MongoDB)**

    - **bash**
    - mvn verify

- **To generate the JaCoCo coverage report along with the tests, use the jacoco profile:**
    - **bash**
    - mvn verify -Pjacoco

- **Run Mutation Testing (PIT)**

    - **bash**
    - mvn pitest:mutationCoverage

- **Build the Docker Image**

    - **bash**
    - mvn package -Pdocker

- **Run the Application Locally To run the Swing application directly from the command line using Maven:**

    - **bash**
    - mvn exec:java -Dexec.mainClass="com.example.attendance.app.AttendanceTrackerApp"

9. Conclusion

The **Student Attendance Tracker** project successfully demonstrates the practical application of modern software engineering principles, moving beyond simple coding to a structured, quality-driven development lifecycle. By combining Test-Driven Development (TDD) with the MVC pattern and automated quality gates, the resulting application is robust, highly maintainable, and ready for production-level deployment.

Furthermore, the integration of Docker and Testcontainers ensures that the application remains portable and reproducible across different development environments. The achievement of 100% code coverage and the passing of all SonarCloud quality gates serve as objective evidence of the project's technical excellence and reliability. Ultimately, this project serves as a comprehensive case study in how automated testing and continuous integration can eliminate technical debt and ensure long-term software stability.